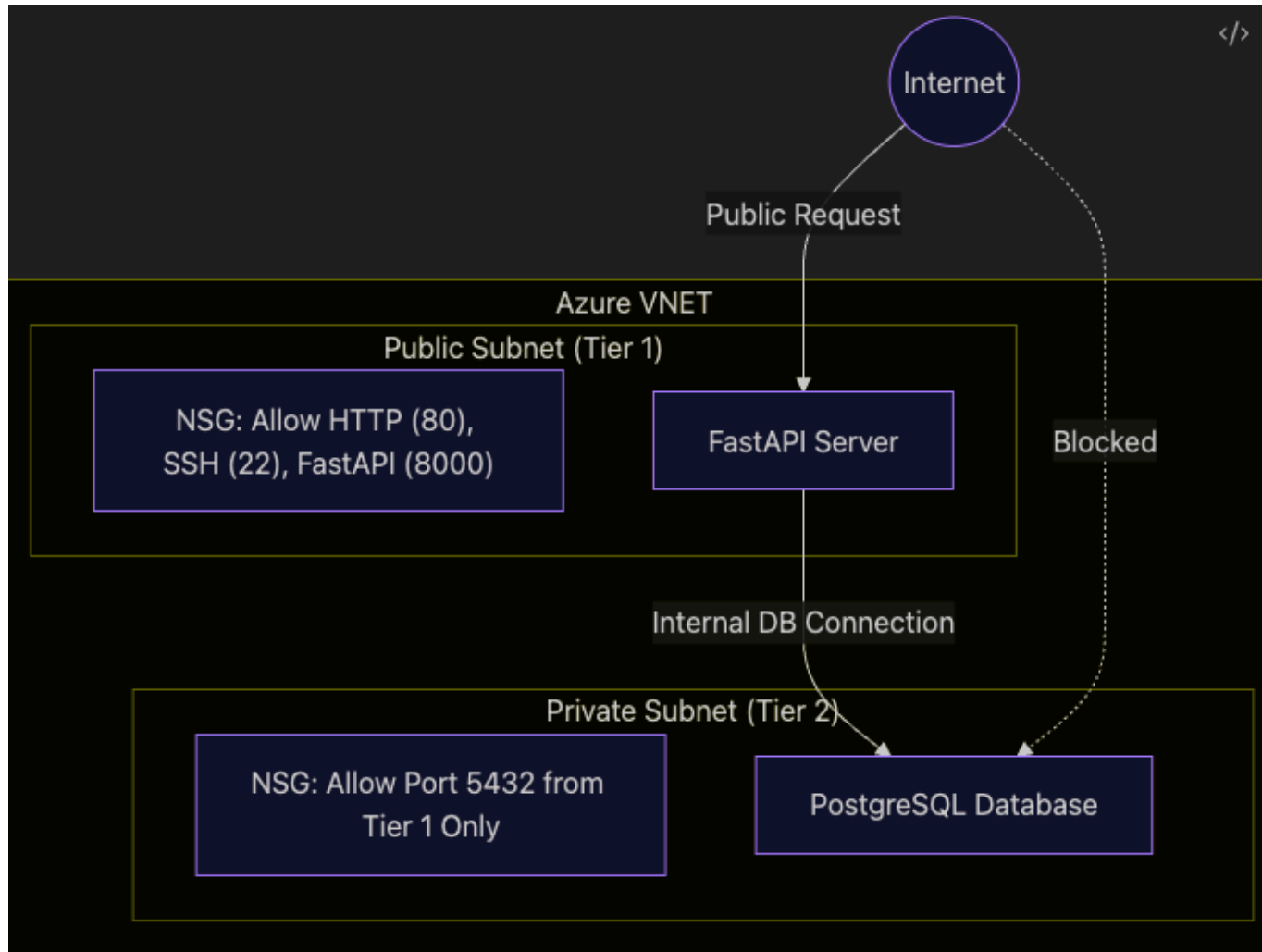


LearningSteps – 2-Tier Azure Deployment

A cybersteps project



Date: 28. March 2026

Participants: Gregor

Project Overview

Today, we deployed the **LearningSteps FastAPI application** to a secure **2-tier architecture** in Microsoft Azure. The focus was on:

- Network segmentation (public vs. private subnets)
- Secure connectivity between the application server and database
- Persistent storage for journal entries
- Running the app in a production-ready manner on a VM

This foundation will support future enhancements such as TLS, reverse proxy, CI/CD pipelines, and observability.

- **Tier 1:** appVM running FastAPI with public IP (port 8000)
- **Tier 2:** dbVM running PostgreSQL, private subnet only
- **NSGs** enforce least-privilege access

Key Steps Completed

Infrastructure Setup

1. Created Azure **Resource Group**: `project_LearningStepsRG`
2. Created **Virtual Network** with:
 - Public Subnet → appVM
 - Private Subnet → dbVM
3. Configured **NSGs**:
 - appVM NSG: SSH (22), HTTP (80), FastAPI (8000)
 - dbVM NSG: PostgreSQL (5432) only from appVM subnet

Database Setup (Tier 2)

1. Deployed PostgreSQL on dbVM without public IP
2. Configured `pg_hba.conf` and `postgresql.conf`:
 - Accept connections only from appVM subnet
 - Require password authentication
3. Created database and user:

```
CREATE DATABASE journalingdb;  
CREATE USER journaluser WITH ENCRYPTED PASSWORD 'StrongPassword123!';  
GRANT ALL PRIVILEGES ON DATABASE journalingdb TO journaluser;
```

4.
 4. Initialized `entries` table and indexes:

```
CREATE TABLE IF NOT EXISTS entries (  
  id VARCHAR PRIMARY KEY,  
  data JSONB NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL  
);
```

```
CREATE INDEX IF NOT EXISTS idx_entries_created_at ON entries(created_at);
CREATE INDEX IF NOT EXISTS idx_entries_data_gin ON entries USING GIN (data);
```

Application Setup (Tier 1)

1. Deployed **FastAPI** on appVM without Docker
2. Installed Python dependencies in a virtual environment:

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

3. Configured `.env` with DB connection string:

```
DATABASE_URL=postgresql://journaluser:StrongPassword123!@10.0.2.4:5432/journalingdb
```

4. Verified DB connectivity from appVM:

```
psql -h 10.0.2.4 -U journaluser -d journalingdb
```

5. Added NSG rule to allow port 8000 for public access:

```
az network nsg rule create \
  --resource-group project_LearningStepsRG \
  --nsg-name app-nsg \
  --name Allow-8000 \
  --priority 120 \
  --access Allow \
  --protocol Tcp \
  --direction Inbound \
  --destination-port-range 8000 \
  --source-address-prefix Internet
```

6. Configured FastAPI as a **systemd service** for automatic start on VM reboot:

```
[Unit]
Description=LearningSteps FastAPI Service
After=network.target
```

```
[Service]
User=azureuser
WorkingDirectory=/home/azureuser/learningsteps
ExecStart=/home/azureuser/learningsteps/venv/bin/uvicorn main:app --host
0.0.0.0 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable learningsteps
sudo systemctl start learningsteps
```

Validation

- API is accessible via **appVM public IP**
- CRUD operations on journal entries work correctly
- PostgreSQL is **not publicly accessible**
- Data persists across VM reboots
- Network segmentation is enforced between Tier 1 and Tier 2

Next Steps / Future Improvements

- Add **TLS/HTTPS** for secure API requests
- Add **NGINX reverse proxy** on port [80/443](#)
- Automate **database migrations** instead of manual SQL
- Implement **CI/CD pipelines** for automated deployments
- Add **monitoring and logging** for observability

Outcome: A secure, production-ready 2-tier IaaS architecture on Azure with FastAPI connected to a private PostgreSQL VM — fully aligned with course objectives on networking, security, and multi-tier design.